**Academic Year: 2026-2027**
**Semester: III**
**Class / Branch: SE IT**
**Subject:ADSAL lab**
**Name of Instructor:prof.Sneha Dalvi**

**Name of Student:Nishant Suryarao**
**Student ID:26204001**
**Date of Performance:08/10/2026**
**Date of Submission:08/10/2026**

---

## Experiment No.09

**Aim:** Implement Floyd Warshall algorithm.

## Code:

```
#include <stdio.h>
#define INF 99999 // large value to represent no edge
void floydWarshall(int V, int graph[V][V]) {
 int dist[V][V], i, j, k;
 // Initialize distance matrix
 for (i = 0; i < V; i++) {
 for (j = 0; j < V; j++) {
 if (graph[i][j] == -1 && i != j)
 dist[i][j] = INF; // no edge
 else
 dist[i][j] = graph[i][j];
 }
 }
 // Main algorithm
 for (k = 0; k < V; k++)
 for (i = 0; i < V; i++)
 for (j = 0; j < V; j++)
 if (dist[i][k] + dist[k][j] < dist[i][j])
 dist[i][j] = dist[i][k] + dist[k][j];
 // Print result
 printf("\nShortest distance matrix:\n");
 for (i = 0; i < V; i++) {
 for (j = 0; j < V; j++) {
 if (dist[i][j] == INF) printf("-1 ");
 else printf("%d ", dist[i][j]);}
 printf("\n");
```

```
 }
}
int main() {
 int V, i, j;
 printf("Enter number of vertices: ");
 scanf("%d", &V);
 int graph[V][V];
 printf("Enter adjacency matrix (use -1 if no edge):\n");
 for (i = 0; i < V; i++)
 for (j = 0; j < V; j++)
 scanf("%d", &graph[i][j]);
 floydWarshall(V, graph);
 return 0;
}
```

**Output Screen Shots:**

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ gcc -o Exp9 Exp9.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ ./Exp9
Enter number of vertices: 3
Enter adjacency matrix (use -1 if no edge):
5 8 66
98 33 44
22 44 77

Shortest distance matrix:
5 8 52
66 33 44
22 30 74
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$
```

**Conclusion :**

The Floyd–Warshall algorithm was successfully implemented to find the shortest paths between all pairs of vertices. The algorithm efficiently updates the distance matrix using dynamic programming and can handle graphs with negative edge weights.

**Conclusion:**